ДЗ4. Отказоустойчивость с Apache ZooKeeper

Вдовина Анастасия Романовна, ИИИ-3-ИнжДан-1-3.
Вариант 7: «Программист, разработчик на Go». 05.11.2026.

1. Подготовка и запуск ансамбля

Подняла три сервера ZooKeeper в Docker на одном компьютере. Использовала образ `zookeeper:3.9.6-jre-17`: ZooKeeper 3.9.6 и Java 17.0.21. Конфигурацию сохранила в `compose.yaml`. Для каждого сервера задала свой `ZOO_MY_ID` и отдельные тома `/data`, `/datalog`. Образ создаёт файл `myid` по указанному идентификатору.

Сервер	myid	Адрес клиента с macOS	Тома данных
zk1	1	127.0.0.1:2181	zk1-data, zk1-datalog
zk2	2	127.0.0.1:2182	zk2-data, zk2-datalog
zk3	3	127.0.0.1:2183	zk3-data, zk3-datalog

Внутри контейнеров порты одинаковые: сетевые пространства и каталоги данных изолированы. Список участников у всех трёх серверов общий:

```
server.1=zk1:2888:3888;2181
server.2=zk2:2888:3888;2181
server.3=zk3:2888:3888;2181
tickTime=2000
initLimit=10
syncLimit=5
```

Запустила ансамбль и проверила роли командой `zkServer.sh status`. В первом запуске zk3 стал лидером, zk1 и zk2 стали последователями.

```
docker compose up -d --wait --wait-timeout 120
docker compose exec -T zk1 zkServer.sh status
docker compose exec -T zk2 zkServer.sh status
docker compose exec -T zk3 zkServer.sh status
```

```
$ cat logs/ensemble/20261105T092042Z/three-alive-zk1-status.log
ZooKeeper JMX enabled by default
Using config: /conf/zoo.cfg
Client port found: 2181. Client address: localhost. Client SSL: false.
Mode: follower

$ cat logs/ensemble/20261105T092042Z/three-alive-zk2-status.log
ZooKeeper JMX enabled by default
Using config: /conf/zoo.cfg
Client port found: 2181. Client address: localhost. Client SSL: false.
Mode: follower

$ cat logs/ensemble/20261105T092042Z/three-alive-zk3-status.log
ZooKeeper JMX enabled by default
Using config: /conf/zoo.cfg
Client port found: 2181. Client address: localhost. Client SSL: false.
Mode: leader
```

Рис. 1. Роли трёх серверов. Просмотр исходных журналов командой cat в терминале.

2. Потеря серверов и эфемерный узел

Остановила zk1: оставшиеся zk2 и zk3 приняли запись и вернули `quorum-survived`. Затем остановила zk2. У одного оставшегося сервера нет большинства: статус не показывает обслуживающую роль, новая запись завершается `ConnectionLoss`. После запуска обоих серверов прочитала сохранённые данные; новым лидером стал zk2.

```
$ grep -E Mode: logs/ensemble/20261105T092042Z/two-alive-zk2-status.log
Mode: follower

$ grep -E Mode: logs/ensemble/20261105T092042Z/two-alive-zk3-status.log
Mode: leader

$ grep -E 'Created /|quorum-survived' logs/ensemble/20261105T092042Z/one-lost-write-read.log
[zk: zk3:2181(CONNECTED) 0] Created /hw4-ensemble-20261105T092042Z/proof
[zk: zk3:2181(CONNECTED) 1] quorum-survived

$ grep -E 'Error contacting service' logs/ensemble/20261105T092042Z/one-alive-zk3-status.log
Error contacting service. It is probably not running.

$ grep -E KeeperErrorCode logs/ensemble/20261105T092042Z/two-lost-write-fails.log
[zk: zk3:2181(CONNECTING) 0] KeeperErrorCode = ConnectionLoss for /hw4-ensemble-20261105T092042Z/without-quorum
```

Рис. 2. Два сервера обслуживают запись; один сервер теряет кворум.

В отдельной сессии `zkCli` создала узел командой `create -e`. Другим клиентом прочитала значение и ненулевой `ephemeralOwner`. После `quit` у владельца повторила чтение: узел исчез.

```
$ grep -E 'Created /' logs/ensemble/20261105T092042Z/ephemeral-creator.log
[zk: zk1:2181(CONNECTED) 0] Created /hw4-ensemble-20261105T092042Z/ephemeral

$ grep -E 'session-owned|ephemeralOwner' logs/ensemble/20261105T092042Z/ephemeral-before-close.log
[zk: zk2:2181(CONNECTED) 0] session-owned
ephemeralOwner = 0x1000020edef0001

$ grep -E 'Node does not exist:' logs/ensemble/20261105T092042Z/ephemeral-after-close.log
[zk: zk2:2181(CONNECTED) 0] Node does not exist: /hw4-ensemble-20261105T092042Z/ephemeral
```

Рис. 3. Создание эфемерного узла, чтение другой сессией и исчезновение после закрытия владельца.

`quit` завершает сессию явно. При потере связи удаление произошло бы после истечения её тайм-аута. Полные команды и выводы сохранила в `logs/ensemble/20261105T092042Z/`.

3. Координаторы и восстановление с чекпоинта

Взяла расчёт пар навыков из ДЗ3. Сохранила без изменений фильтр Go/Golang, нормализацию навыков, разбиение CSV и map-функцию из коммита `795746e`. В ДЗ3 читала CSV из S3; для этих опытов использовала локальную копию того же полного файла: 847 619 384 байта, 229 388 записей. Перед каждым запуском проверила SHA-256 источника:

```
303db922b541e4440906f5a7e8be32fb55e8808f950ffddb8c272f1548212a65
```

Запустила два отдельных процесса координатора A и B. Оба подключаются к трём серверам с тайм-аутом сессии 10 секунд; клиентская библиотека – Kazoo 2.11.0, Python 3.14.4, macOS arm64. Кандидаты создают эфемерные последовательные узлы в `/election`. Узел с наименьшим номером получает лидерство, остальные следят за своим предшественником. При продвижении в лидеры транзакция проверяет существование узла кандидата и увеличивает эпоху. Так процесс с уже потерянной сессией не получает новую эпоху.

Что сохраняется в чекпоинте

Map обрабатывает чанки по 2 000 записей четырьмя процессами. Reduce применяет результаты в порядке чанков. В чекпоинт записала частоты всех пар, число обработанных и отобранных записей, номер следующего чанка, размер чанка и контрольную сумму исходного CSV.

Файл снимка сохраняется через временный файл и атомарное переименование. После этого транзакция с проверкой эпохи обновляет указатель `/checkpoint`. При восстановлении проверяются SHA-256 снимка и совместимость источника. В обоих режимах защита чекпоинта включена: бывший лидер не может заменить состояние нового координатора.

Как воспроизвела отказ

После восьми сохранённых чанков приостановила A сигналом `SIGSTOP`. Вместе с расчётом остановилась отправка heartbeat. После истечения сессии B стал лидером с эпохой 2, загрузил чекпоинт и продолжил с индекса 8 (девятого чанка). Уже учтённый префикс повторно не прибавлялся.

Дождалась завершения B и паузы A не менее 40 секунд, затем отправила A `SIGCONT`. Задержка 0,12 секунды между применёнными чанками позволяет стабильно приостановить расчёт до завершения. Частоты от неё не меняются. Для демонстрации устаревшей записи бывший лидер продолжает локальный расчёт и пытается опубликовать итог даже после потери своей сессии.

Как измерила время и проверила результат

Время смены координатора посчитала по монотонным часам:

$$\Delta t = t(\texttt{leader у B}) - t(\texttt{SIGSTOP для A}).$$

Измеряется получение лидерства запасным координатором. Ожидание готового результата в этот интервал не входит. Сигналы, PID и исходные метки сохранила в `control.json`, события координаторов – в `A.log` и `B.log`.

Для A и B в каждом опыте сравнила все пары и их частоты с эталоном ДЗ3. Эталон содержит 3 123 отобранные вакансии, 13 552 уникальные пары и 61 694 вхождения. Во всех четырёх файлах-кандидатах число расхождений равно нулю.

4. Приостановка лидера без защиты: FENCING=0

Запустила опыт с отключённой проверкой эпохи при публикации результата:

```
.venv/bin/python scripts/failover_demo.py --mode 0
```

После приостановки A координатор B стал лидером через **11,006 с**, восстановил восемь сохранённых чанков и завершил расчёт до возобновления A. Пауза A составила 40,058 с.

В журнале B появилась строка «записал результат» с эпохой 2 и версией указателя 1. После `SIGCONT` A тоже закончил расчёт и записал результат с устаревшей эпохой 1. Получилось **две принятые публикации**; версия `/result` стала **2**, окончательный указатель принадлежит A. Совпадение чисел не устраняет ошибку: право на публикацию у A уже потеряно.

```
FENCING=0; selected fields from saved A.log, B.log, summary.json

B.log:
{"time": "2026-11-05T09:51:07.281741+00:00", "coordinator": "B", "event": "leader", "epoch": 2, "next_chunk": 8, "restored_from_epoch": 1}

B.log:
{"time": "2026-11-05T09:51:26.449126+00:00", "coordinator": "B", "event": "published", "epoch": 2, "pointer_version": 1, "message": "записал результат"}

A.log:
{"time": "2026-11-05T09:51:53.473169+00:00", "coordinator": "A", "event": "published", "epoch": 1, "pointer_version": 2, "message": "записал результат"}

logs/failover/20261105T095054Z-without-fencing/summary.json:
session_timeout_ms = 10000
leader_switch_seconds = 11.006047084
pause_seconds = 40.058195209
backup_resumed_next_chunk = 8
backup_finished_before_resume = true
published_count = 2
pointer_version = 2
old_leader_error = null
pair_differences_from_hw3 = {"A": 0, "B": 0}
passed = true
pointer = {"coordinator": "A", "epoch": 1, "selected_rows": 3123, "unique_pairs": 13552, "pair_occurrences": 61694}
```

Рис. 4. Фрагменты A.log, B.log и выбранные поля summary.json: обе публикации приняты, версия указателя 2.

Сверила оба результата с ДЗ3: 3 123 вакансии, 13 552 пары, 61 694 вхождения, расхождений частот – 0. Полные журналы сохранила в `logs/failover/20261105T095054Z-without-fencing/`.

5. Приостановка лидера с защитой: FENCING=1

Повторила опыт с проверкой эпохи при публикации:

```
.venv/bin/python scripts/failover_demo.py --mode 1
```

В стал лидером через **11,833 с**, восстановился с того же префикса из восьми чанков и опубликовал результат с эпохой 2. Пауза A составила 40,066 с. После возобновления A завершил локальные вычисления, но его публикация со старой эпохой 1 получила `BadVersionError`.

Получилась **одна принятая публикация**: указатель версии **1** остался от B. Файлы-кандидаты сохраняются отдельно по эпохам. Наличие файла A само по себе не делает его официальным результатом: читатели используют только принятый указатель `/result`.

```
FENCING=1; selected fields from saved A.log, B.log, summary.json

B.log:
{"time": "2026-11-05T09:52:07.269059+00:00", "coordinator": "B", "event": "leader", "epoch": 2, "next_chunk": 8, "restored_from_epoch": 1}

B.log:
{"time": "2026-11-05T09:52:26.571357+00:00", "coordinator": "B", "event": "published", "epoch": 2, "pointer_version": 1, "message": "записал результат"}

A.log:
{"time": "2026-11-05T09:52:52.274696+00:00", "coordinator": "A", "event": "publication_rejected", "epoch": 1, "error": "BadVersionError", "message": "BadVersionError: устаревша
я эпоха, запись отклонена"}

logs/failover/20261105T095153Z-with-fencing/summary.json:
session_timeout_ms = 10000
leader_switch_seconds = 11.83265675
pause_seconds = 40.066117833
backup_resumed_next_chunk = 8
backup_finished_before_resume = true
published_count = 1
pointer_version = 1
old_leader_error = "BadVersionError"
pair_differences_from_hw3 = {"A": 0, "B": 0}
passed = true
pointer = {"coordinator": "B", "epoch": 2, "selected_rows": 3123, "unique_pairs": 13552, "pair_occurrences": 61694}
```

Рис. 5. Публикация B принята; попытка A отклонена с BadVersionError. Версия указателя осталась 1.

Оба вычисленных набора частот снова совпали с ДЗ3: 3 123 вакансии, 13 552 пары, 61 694 вхождения, расхождений – 0. Журналы сохранила в `logs/failover/20261105T095153Z-with-fencing/`.

6. Сравнение и ответы на вопросы

Проверка	FENCING=0	FENCING=1
Тайм-аут сессии	10 с	10 с
Смена координатора A → B	11,006 с	11,833 с
Пауза A	40,058 с	40,066 с
Восстановленный префикс	8 чанков	8 чанков
B завершил до возобновления A	да	да
Принятых публикаций	2	1
Версия указателя результата	2	1
Автор окончательного указателя	A	B
Ошибка публикации A	нет	BadVersionError
Расхождений с ДЗ3 у A / B	0 / 0	0 / 0

Почему серверов три, а не два или четыре?

Для работы нужно строгое большинство. В ансамбле из двух серверов нужны оба, поэтому потеря одного сразу останавливает запись. В ансамбле из трёх достаточно двух: один сервер можно потерять. Для четырёх нужно три, поэтому переносится всё ещё только один отказ, хотя серверов больше. Я использовала минимальный ансамбль, который сохраняет работу после потери одного участника. Контейнеры находятся на одном компьютере: этот стенд показывает отказы процессов, а отказ самого компьютера остановил бы сразу весь ансамбль.

Почему эпоху проверяет получатель записи, а не ZooKeeper?

ZooKeeper хранит состояние координации, но не может остановить зависший процесс и отменить его запрос к другому хранилищу. После возобновления бывший лидер может отправить уже устаревшую запись. Поэтому получатель должен проверить эпоху атомарно с изменением данных. Проверка только перед отправкой запроса оставила бы промежуток для смены лидера.

В моём стенде официальный результат опубликован указателем в самом ZooKeeper. Проверку версии `/epoch` и изменение `/result` объединила в одну транзакцию, поэтому здесь получатель и есть ZooKeeper. Для прямой записи в S3 или БД такую защиту нужно реализовать на стороне этого хранилища либо принимающего запись сервиса.

Файлы и воспроизведение

Конфигурация – `compose.yaml`; полные журналы – `logs/`; эталон – `baseline/`; четыре набора частот – `results/`; исходные скриншоты – `screenshots-terminal/`. В `README.md` сохранила команды ручного запуска двух координаторов. Проверила 13 тестов: целостность снимка, источник и размер чанка, повтор/пропуск чанка, защиту эпохи и потерю кандидата перед продвижением. Результат – `13 passed`, полный вывод – `logs/tests.log`.

```
.venv/bin/python scripts/ensemble_demo.py
.venv/bin/python scripts/failover_demo.py --mode both
.venv/bin/python -m pytest tests -q
```